KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Vehicle Rental Booking and Payment Management System

Submitted by **KASHINI K**

25BCS125

Under the Guidance of

Mr.ARAVIND

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026 APRIL 2026**

Semester: **II**

DECLARATION

I hereby declare that the project entitled "Vehicle Rental Booking and Payment Management System" submitted in partial fulfillment of the requirements for the course "Java Programming" is a **record of my original work** carried out under the guidance of **Mr. Aravind**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

KASHINI.K

CERTIFICATE

This is to certify that the project entitled **"Vehicle Rental Booking and Payment Management System"** is a **bona fide work** carried out by **KASHINI.K(I-B.SC.CS-'B'(2025)),KG College of Arts & Science**, in partial fulfillment of the requirements for the course **Java Programming**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place:Coimbatore

Date:10-04-2026

Submitted for the Viva-Voce Examination held on 10-04-2026

Guide:
Mr.Aravind

(Technical Trainer, KGCAS)

Head of the Department
Dr.Boopalan
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	6
1.1	Objective	6
1.2	Overview	6
1.3	Features	7
2	**Program Analysis**	8
2.1	Problem Definition	8
2.2	Project Overview	8
2.3	Scopes and Limitations	9
3	**System Design**	10
3.1	Modules	10
4	**Methodology**	11
4.1	Problem Analysis	11
4.2	System Design	11
4.3	Program Development	11
4.4	Testing and Validation	11
4.5	Result and Documentation	11
4.6	Algorithm	12
4.7	Flowchart	13
5	**Source Code**	15
6	**Input and Output Screens**	23
7	**Discussion and Conclusion**	25
7.1	Discussion	25
7.2	Conclusion	25
8	**References**	26

ABSTRACT

The Vehicle Rental System and Payment Management System is a Java-based desktop application designed to automate the process of vehicle booking and payment tracking in small and medium-scale rental businesses. Traditionally, vehicle rental operations are handled manually using registers or spreadsheets, which often results in data inconsistency, calculation errors, poor record maintenance, and difficulty in tracking bookings and payments. This project aims to provide a structured and efficient digital solution using Java Swing to simplify rental management. The system includes login authentication to ensure secure access to authorized users. After successful login, users can enter customer details such as name, mobile number, vehicle type, model, booking date, and rental duration. The application automatically calculates the rental amount based on the selected vehicle and duration, reducing manual calculation errors. The payment management feature allows users to select payment mode, track payment status as paid or unpaid, and maintain proof of payment details. All booking information is stored within the system using appropriate data structures, ensuring organized record maintenance. Additionally, a calendar view is provided to display bookings date-wise, helping users easily identify reserved dates and avoid scheduling conflicts. The system also supports viewing booking details by clicking on specific dates. This application improves operational efficiency, reduces paperwork, enhances accuracy, and provides better control over rental and payment management. The project demonstrates the use of Java Swing components, event-driven programming, and collections framework to develop a real-world application that is simple, reliable, and suitable for small vehicle rental businesses.

1. INTRODUCTION

In the vehicle rental industry, managing bookings manually can be time-consuming and error-prone. Small rental businesses often rely on handwritten records or basic tools, which leads to inefficiency and poor data management.

With the advancement of technology, automated systems are essential to improve service quality and maintain proper records.

The **Vehicle Rental Booking System** is a Java-based application that automates booking, billing, and record management. The system allows users to:

- Log in securely
- Enter customer and vehicle details
- Calculate rental cost automatically
- Store booking data
- View bookings using a calendar interface

This project uses **Java Swing for GUI** and demonstrates real-world application of programming concepts.

1.1 Objective

The objective of this project is to develop an efficient and user-friendly system that simplifies the process of vehicle rental booking

- To automate booking and billing process
- To reduce manual errors
- To provide user-friendly interface
- To manage bookings efficiently
- To display bookings using a calendar system

1.2 Overview

The system is a GUI-based application where users:

- Enter customer details
- Select vehicle type (Bike, Car, Van)
- Choose model and rental duration
- Calculate total amount automatically
- Save booking details

Bookings are stored and can be viewed using a **calendar interface**, where booked dates are highlighted.

1.3 Features

The Vehicle Rental Booking System includes several important features that make the booking process simple, efficient, and user-friendly:

- **Login Authentication** – Allows only authorized users to access the system.
- **Customer Details Entry** – Stores customer name and mobile number.
- **Mobile Number Validation** – Ensures mobile number contains exactly 10 digits.
- **Vehicle Type Selection** – User can choose Bike, Car, or Van.
- **Dynamic Model Selection** – Displays models based on selected vehicle type.
- **Rental Duration Input** – User enters number of rental hours.
- **Automatic Cost Calculation** – Calculates amount based on hours and vehicle type.
- **Payment Mode Selection** – Supports Cash and Online payment options.
- **Proof Type Selection** – Allows selection of ID proof (Aadhar/License).
- **PDF Upload Feature** – Uploads customer proof document.
- **Payment Status Tracking** – Marks booking as Paid or Unpaid.
- **Booking Data Storage** – Saves booking details in memory.
- **Calendar View** – Displays bookings according to selected date.
- **Booked Date Highlighting** – Shows booked days in calendar.
- **Booking Details Display** – Shows customer and vehicle details on click.
- **PDF Opening Option** – Opens uploaded document directly.
- **User-Friendly GUI** – Simple interface using Java Swing.

2. PROGRAM ANALYSIS

Program analysis is an important stage in software development where the problem is studied and the requirements of the system are identified. This phase helps in understanding the limitations of the existing system and designing a computerized solution to improve efficiency.

In the Vehicle Rental Booking System, the analysis focuses on identifying the issues faced in manual vehicle booking and providing an automated solution. The system is designed to manage customer details, vehicle selection, booking dates, and rental cost calculation in a structured manner.

The program analysis also helps in defining the functionalities required for the application, such as login authentication, booking management, payment tracking, and calendar-based booking display

2.1 Problem Definition

In many small-scale vehicle rental businesses, booking information is maintained manually using notebooks or registers. This manual system creates several problems and inefficiencies in managing bookings.

Some of the common problems include:

- Errors in calculating rental charges manually
- Difficulty in tracking bookings for different dates
- Lack of proper customer record management
- Time-consuming booking process
- No proper method to track payment status
- Difficulty in managing multiple bookings
- Risk of losing customer proof documents

Manual systems also make it difficult to check availability of vehicles for a particular date. This may result in double bookings and customer dissatisfaction.

Therefore, there is a need for a computerized system that can automate the booking process, calculate rental charges automatically, and maintain booking records efficiently.

The Vehicle Rental Booking System is designed to solve these problems by providing an easy-to-use application for managing vehicle rentals.

2.2 Project Overview

The Vehicle Rental Booking System is a Java-based application that automates the process of vehicle booking. The system provides a graphical user interface where users can enter customer details, select vehicle type, choose model, and specify rental duration.

The system automatically calculates the rental amount based on the selected vehicle and number of hours. It also allows users to upload customer proof documents in PDF format and track payment status.

The system stores booking details and displays them using a calendar interface. Booked dates are highlighted, and users can click on a specific date to view detailed booking information.

Main functionalities of the system include:

- User login authentication
- Customer details entry
- Vehicle type and model selection
- Rental cost calculationPayment mode selection
- PDF document upload
- Booking data storage
- Calendar-based booking display

This system improves efficiency and reduces manual effort in managing vehicle rental bookings.

2.3 Scopes and Limitations

2.4 Scope

Scope:

- Small vehicle rental businesses
- Travel agencies
- Personal rental services

Limitations:

- No permanent database storage
- Single user access
- Basic GUI design

3. SYSTEM DESIGN

System design is the phase where the structure and working of the system are planned before implementation. In the Vehicle Rental Booking System, the application is divided into different modules to perform specific tasks. This modular design helps in organizing the program and improving efficiency.

The system follows a simple workflow where the user logs in, enters booking details, calculates rental amount, saves the booking, and views bookings using a calendar interface.

3.1Modules

Login Module
This module verifies the username and password entered by the user. Only valid users are allowed to access the booking system.

Booking Module
This module collects customer details such as name, mobile number, vehicle type, model, and booking date.

Calculation Module
This module calculates the rental amount automatically based on the vehicle type and number of hours.

Storage Module
This module stores booking details using Java collections. Multiple bookings can be saved and retrieved when required.

Calendar Module
This module displays bookings using a calendar view. Booked dates are highlighted, and users can click a date to view booking details.

4. METHODOLOGY

Methodology describes the steps followed to develop the Vehicle Rental Booking System. A structured approach was used to design, implement, and test the application. The development process includes problem analysis, system design, coding, testing, and documentation.

4.1Problem Analysis

In this phase, the issues in manual vehicle booking systems were identified. Problems such as manual calculation errors, poor record management, and difficulty in tracking bookings were studied. Based on these issues, requirements for an automated booking system were defined.

4.2System Design

The system structure was planned by dividing the application into modules such as login, booking, calculation, storage, and calendar. This helped in organizing the development process.

4.3Program Development

The application was developed using Java programming language and Java Swing for graphical interface. Core Java concepts such as classes, objects, collections, and event handling were used to implement system functionalities.

4.4Testing and Validation

The system was tested with different inputs to ensure correct functionality. Test cases were used to verify login, booking entry, calculation, and calendar display.

4.5Result and Documentation

After successful testing, the results were documented. Screenshots of the system and explanation of functionalities were included to demonstrate the working of the application.

4.6. Algorithm

1. Start the program.
2. Display the login page.
3. Enter username and password.
4. Validate login credentials.
5. If login is valid, open booking form; otherwise display error message.
6. Enter customer name and mobile number.
7. Select vehicle type.
8. Select vehicle model.
9. Enter number of rental hours.
10. Enter booking date.
11. Select payment mode.
12. Select proof type.
13. Upload proof document (PDF).
14. Click calculate button to compute rental amount.
15. Mark payment status (Paid/Unpaid).
16. Save booking details.
17. Store booking data in system.
18. Open calendar view.
19. Highlight booked dates.
20. Click a date to view booking details.
21. Display customer booking information.
22. End the program.

4.7. Flowchart

```
Start
  |
  v
Login Page
  |
  v
Enter Username & Password
  |
  v
Is Login Valid?
  | Yes                    | No
  v                        v
Booking Form             Show Error Message
  |                        |
  v                        |
Enter Customer Details <-----
  |
  v
Select Vehicle Type & Model
  |
  v
Enter Rental Hours & Date
  |
  v
Calculate Amount
  |
  v
```

Upload Proof & Select Payment

|

v

Save Booking

|

v

Store Booking Data

|

v

Open Calendar View

|

v

Select Date

|

v

Display Booking Details

|

v

En

5. SOURCE CODE

```
import javax.swing.*;
import java.awt.*;  import
java.io.File; import
java.awt.Desktop; import
java.util.*;
public class App {public static void main(String[] args) { SwingUtilities.invokeLater(LoginPage::new);
    }
}
class LoginPage extends JFrame
    { JTextField username;
    JPasswordField password;
    LoginPage() {
        setTitle("Login");
        setSize(300,150);
        setLayout(new GridLayout(3,2));
        add(new JLabel("Username"));
        username = new JTextField();
        add(username);
        add(new JLabel("Password")); password
        = new JPasswordField(); add(password);
       JButton login = new JButton("Login"); add(new
        JLabel());
        add(login);

        login.addActionListener(e -> { if(username.getText().equals("Kashini") &&
              String.valueOf(password.getPassword()).equals("vehiclerental")){ new
               BookingForm();
               dispose();
            } else {
               JOptionPane.showMessageDialog(this,"Invalid  login");
            }
```

```
        });
       setVisible(true);
    }
}
class BookingStore {
    static Map<String, java.util.List<Booking>> data = new HashMap<>();
}
class Booking
    { String
    details; String
    pdf; boolean
    paid;
    Booking(String d,String p,boolean paid){
        this.details=d;
        this.pdf=p;
        this.paid=paid;
    }
}
class BookingForm extends JFrame
    { JComboBox<String> type,model,payment,proof;
    JTextField hours,date,name,mobile;
    JLabel amount,fileLabel;
    JCheckBox paid;
    String pdfPath="";
    BookingForm(){
       setTitle("Booking");
        setSize(420,550);
        setLayout(new GridLayout(14,2));
       add(new JLabel("Customer Name"));
       name=new JTextField("Kashini");
```

```
 add(name);
add(new JLabel("Mobile Number"));
 mobile=new JTextField(); add(mobile);
add(new JLabel("Vehicle Type"));
 type=new JComboBox<>(new String[]{"Bike","Car","Van"}); add(type);
add(new JLabel("Model"));
 model=new JComboBox<>();
 add(model);
add(new JLabel("Hours"));
 hours=new JTextField();
 add(hours);
 add(new JLabel("Date (yyyy-mm-dd)"));
 date=new JTextField();
 add(date);
add(new JLabel("Payment"));
 payment=new JComboBox<>(new String[]{"Cash","Online"});
 add(payment);

 add(new JLabel("Proof Type"));
 proof=new JComboBox<>(new String[]{"Aadhar","License"}); add(proof);
 JButton upload=new JButton("Upload PDF");
 add(upload);
 fileLabel=new JLabel("No file"); add(fileLabel);
paid=new JCheckBox("Paid");
 add(paid);
```

```
        add(new JLabel("Amount"));
         amount=new JLabel("0");
         add(amount);
        JButton calc=new JButton("Calculate");
         JButton save=new JButton("Save");
         add(calc); add(save);
        JButton calendar=new JButton("Calendar");
         add(calendar);
         type.addActionListener(e->loadModel()); calc.addActionListener(e-
         >calculate()); upload.addActionListener(e->upload());
         save.addActionListener(e->save());  calendar.addActionListener(e->{
         new CalendarUI(); dispose();});
        loadModel();
         setVisible(true);
    }
     void loadModel(){   model.removeAllItems();
         if(type.getSelectedItem().equals("Bike")){
             model.addItem("Activa");  model.addItem("Pulsar");
         }else if(type.getSelectedItem().equals("Car"))
             { model.addItem("Swift"); model.addItem("Innova");
         }else{
             model.addItem("Traveller");
         }
    }
void calculate(){
         try{
             int h=Integer.parseInt(hours.getText());
```

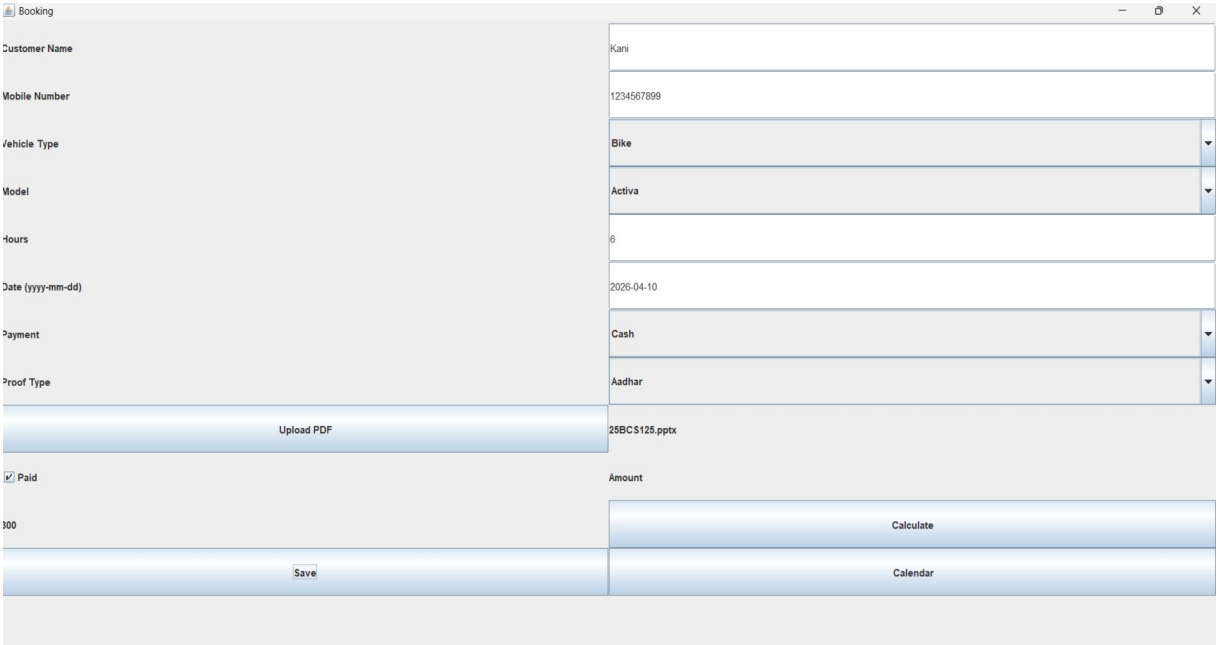
```
            int r= type.getSelectedItem().equals("Bike")?50: type.getSelectedItem().equals("Car")?100:150;
            amount.setText(""+(h*r));
        }catch(Exception ignored){}
    }
  void upload(){
        JFileChooser fc=new JFileChooser();
        if(fc.showOpenDialog(this)==JFileChooser.APPROVE_OPTION){
            pdfPath=fc.getSelectedFile().getAbsolutePath(); fileLabel.setText(fc.getSelectedFile().getName());
        }
    }
 void save(){
String phone = mobile.getText(); if(!phone.matches("\
 \d{10}")){
            JOptionPane.showMessageDialog(this,"Mobile number must be exactly 10 digits"); return;
        }

        calculate();

        String key=date.getText(); String
       details =
                "Customer: "+name.getText()+"\n"+ "Mobile:
                "+mobile.getText()+"\n"+ "Vehicle:
                "+type.getSelectedItem()+"\n"+ "Model:
                "+model.getSelectedItem()+"\n"+ "Hours:
                "+hours.getText()+"\n"+ "Amount:
                "+amount.getText()+"\n"+
```

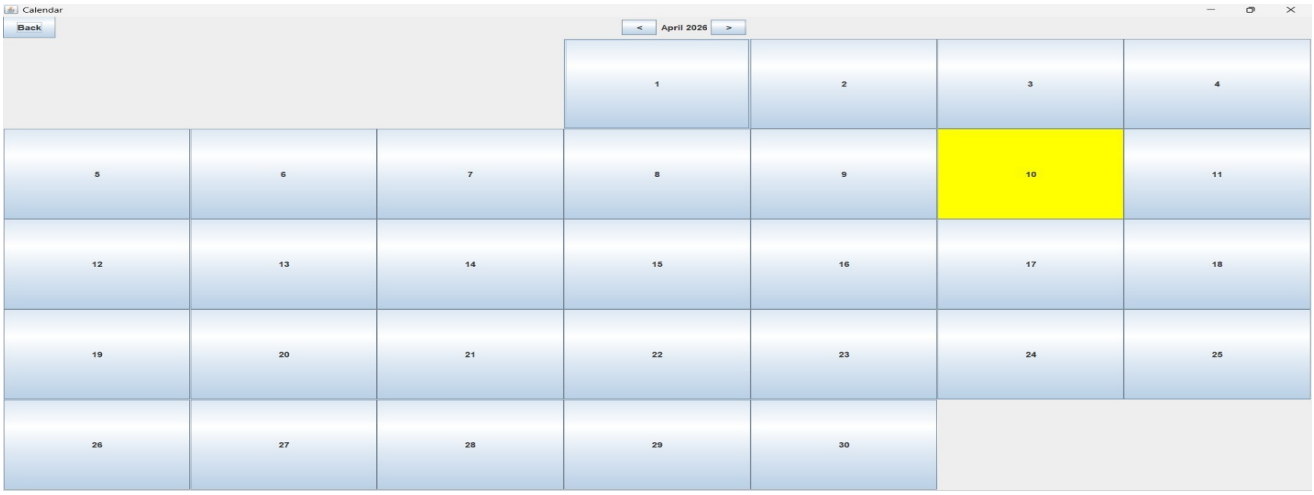
```
                "Payment: "+payment.getSelectedItem()+"\n"+
                "Proof: "+proof.getSelectedItem();

        Booking b=new Booking(details,pdfPath,paid.isSelected());
        BookingStore.data.computeIfAbsent(key,k->new ArrayList<>()).add(b);
        JOptionPane.showMessageDialog(this,"Saved");
    }
}
class CalendarUI extends JFrame { Calendar
    cal=Calendar.getInstance();
    JPanel panel=new JPanel(new GridLayout(0,7)); JLabel
   title=new JLabel("",SwingConstants.CENTER);
   CalendarUI(){
        setTitle("Calendar");
        setSize(700,500);  setLayout(new
        BorderLayout());
       JPanel top=new JPanel(new BorderLayout());
       JButton back=new JButton("Back");
        JButton prev=new JButton("<");
        JButton next=new JButton(">");

        JPanel nav=new JPanel();
        nav.add(prev); nav.add(title);
        nav.add(next);
        top.add(back,BorderLayout.WEST);
       top.add(nav,BorderLayout.CENTER);
       add(top,BorderLayout.NORTH);
       add(panel,BorderLayout.CENTER);
```

```
    back.addActionListener(e->{ new BookingForm(); dispose();});
    prev.addActionListener(e->{ cal.add(Calendar.MONTH,-1); refresh();});
    next.addActionListener(e->{ cal.add(Calendar.MONTH,1); refresh();});
    refresh();
     setVisible(true);
 }
void refresh(){
     panel.removeAll();
    int month=cal.get(Calendar.MONTH); int
     year=cal.get(Calendar.YEAR);
      title.setText(new java.text.SimpleDateFormat("MMMM yyyy").format(cal.getTime())); Calendar
     temp=(Calendar)cal.clone();
     temp.set(Calendar.DAY_OF_MONTH,1);
    int start=temp.get(Calendar.DAY_OF_WEEK)-1;
     int days=temp.getActualMaximum(Calendar.DAY_OF_MONTH); for(int
 i=0;i<start;i++) panel.add(new Jlabel(""));
for(int i=1;i<=days;i++){
         JButton b=new JButton(""+i);
         String  key=String.format("%04d-%02d-%02d",year,month+1,i);

            if(BookingStore.data.containsKey(key))
              b.setBackground(Color.YELLOW);
             b.addActionListener(e->show(key));
          panel.add(b);
     }
     panel.revalidate();
     panel.repaint();
  }
   void show(String key){
```

```
        java.util.List<Booking> list=BookingStore.data.get(key);
        if(list==null){
            JOptionPane.showMessageDialog(this,"No booking");
            return;
        }
  StringBuilder sb=new StringBuilder(); for(int
  i=0;i<list.size();i++){
            Booking b=list.get(i);
            sb.append("Booking ").append(i+1).append("\n");
            sb.append(b.details).append("\n");
            sb.append("Status: ").append(b.paid?"PAID":"UNPAID").append("\n");
            sb.append("-----------------------\n");
        }
JTextArea area=new JTextArea(sb.toString());
        area.setEditable(false);
 int
                option=JOptionPane.showOptionDial
                og( this,new JScrollPane(area),
                "Customer Details",
                JOptionPane.DEFAULT_OPTION,
                JOptionPane.INFORMATION_MESSAGE,
                null,new String[]{"Open PDF","Close"},"Close"); if(option==0){
            try{
                Desktop.getDesktop().open(new  File(list.get(0).pdf));
            }catch(Exception ignored){}
        }
    }
```

6. INPUT AND OUTPUT SCREENS




Calendar
Back
<
April 2026
>
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30

7. DISCUSSION & CONCLUSION

7.1 Discussion

The Vehicle Rental Booking System was developed to simplify and automate the process of vehicle booking and management. The system allows users to log in, enter customer details, select vehicle type and model, and calculate the rental amount automatically. The inclusion of payment mode selection and proof document upload helps maintain proper customer verification records.

During the development of the project, various core Java concepts such as classes, objects, event handling, and collections were implemented. The graphical user interface was designed using Java Swing, which makes the system easy to use and interactive. The calendar feature helps users view bookings date-wise and prevents confusion in managing reservations.

The system was tested with different inputs to verify that booking details are stored correctly and rental calculations are accurate. The results show that the application reduces manual work, improves accuracy, and makes booking management easier. The ability to highlight booked dates and view detailed information improves usability.

Overall, the system performs efficiently for small-scale vehicle rental operations and demonstrates practical implementation of Java programming.

7.2Conclusion

The Vehicle Rental Booking System is a simple and effective application designed to automate the vehicle rental process. The system reduces manual effort, minimizes calculation errors, and helps maintain organized booking records. The calendar-based booking display provides an easy way to track reservations.

The project successfully achieves its objectives by providing login authentication, booking management, automatic calculation, and booking visualization features. The system is user-friendly and suitable for small rental businesses.

This project also demonstrates how Java Swing and core Java concepts can be used to develop real-world applications. In the future, the system can be enhanced by adding database support, online payment integration, and improved user interface for better performance.

8. REFERENCES

REFERENCES

1. E. Balagurusamy, *Programming with Java*, McGraw-Hill Education.
2. Herbert Schildt, *Java: The Complete Reference*, Oracle Press.
3. Oracle Corporation, *Java Documentation*, https://docs.oracle.com/javase/
4. Cay S. Horstmann, *Core Java Volume I – Fundamentals*, Pearson Education.
5. Java Swing Tutorial, https://docs.oracle.com/javase/tutorial/uiswing/
6. Online Java Programming Tutorials and examples.